

Git Journey — Study Notes

Hands-on notes from the `git-journey` repo · Görkem Utku Kaymaz · 21 June 2026

These notes are built from real terminal sessions in `C:\Users\gutkan_zfmr6br\Desktop\git-journey`. They cover the full workflow you practised — repo setup, branching, merging, conflict resolution, and syncing with GitHub — plus dedicated sections on **revert**, **rebase**, and **pull requests**.

Contents

1. Git Fundamentals & the Three Areas
2. Starting a Repository — `init` & `clone`
3. Tracking Changes — `status`, `add`, `commit`
4. Undoing & Removing — `reset`, `rm`, `restore`
5. Branching & `checkout`
6. Merging & Resolving Conflicts
7. Inspecting History — `log`, `diff`, detached HEAD
8. Working with Remotes — `push`, `fetch`, `pull`
9. Pull Requests
10. `revert` — Undo a Commit Safely
11. `rebase` — Rewrite History Cleanly
12. Quick Command Cheat-Sheet
13. Mistakes Logged & Lessons Learned

1. Git Fundamentals & the Three Areas

Git is a **distributed version control system**. Every change you make moves through three areas:

Area	What it is	Move into it with
Working Directory	Your actual files on disk that you edit.	(editing files)
Staging Area (Index)	A snapshot of what will go into the next commit.	<code>git add</code>
Repository (.git)	Permanent committed history.	<code>git commit</code>

Mental model: `add` selects *what* to save → `commit` saves it → `push` shares it with the remote (GitHub).

2. Starting a Repository — `init` & `clone`

Create a brand-new repo

```
mkdir git-one
cd git-one
```

```
git init          # creates the hidden .git/ folder
git status        # On branch master / No commits yet
```

Windows note: `touch one.txt` does *not* work in CMD (`'touch'` is not recognized). Use `echo. > one.txt` to create an empty file, or `type nul > one.txt` . `ls` also fails in CMD — use `dir` .

Copy an existing repo from GitHub

```
git clone https://github.com/GorkemUtku02/git-journey.git
```

Clone downloads the entire history and automatically sets up a remote named `origin` pointing at the GitHub URL.

Watch the typos you hit: `gti clone` and `git chechkout` are not commands. Git helpfully suggests the closest match ("The most similar command is checkout").

3. Tracking Changes — `status`, `add`, `commit`

Command	Purpose
<code>git status</code>	Shows staged, unstaged, and untracked files. Run it constantly.
<code>git add <file></code>	Stage one file.
<code>git add .</code>	Stage everything <i>in/below the current folder</i> .
<code>git add -A</code> / <code>--all</code>	Stage everything in the whole repo (adds, edits, deletes).
<code>git add *.txt</code>	Stage by shell wildcard (only matching names).
<code>git commit -m "msg"</code>	Save the staged snapshot with a message.

```
git add .
git commit -m "I created three.txt"
# [development aa58ca5] I created three.txt
# 1 file changed, 1 insertion(+)
```

Folder matters with `git add .` — when you ran it from inside `myFolder` , it only staged `three.txt` there and left `../one.txt` unstaged. `git add .` is relative to your current directory; `git add -A` is not.

Set your identity once (otherwise commits have wrong/missing author):

```
git config --global user.name "GorkemUtku02"
git config --global user.email "gutkukaymaz@gmail.com"
```

4. Undoing & Removing — `reset`, `rm`, `restore`

Command	Effect	Touches your files?
<code>git reset (= --mixed)</code>	Unstages files; keeps your edits.	No — safe

<code>git reset --soft <commit></code>	Moves HEAD back, keeps changes <i>staged</i> .	No
<code>git reset --hard</code>	Throws away all uncommitted changes back to last commit.	Yes — destructive
<code>git rm <file></code>	Deletes file <i>and</i> stages the deletion.	Yes
<code>git rm --cached <file></code>	Stops tracking but keeps the file on disk.	No
<code>git rm -r <folder></code>	Recursively remove a folder.	Yes

```
git add -A
git reset          # Unstaged changes after reset: M one.txt, M two.txt
git reset --hard   # HEAD is now at 6f09646 ... (wipes working changes)
```

From your log: `git rm four.txt` failed with “local modifications” — Git protects you. You then used `git rm -f four.txt` (force) or `--cached`. Prefer `--cached` when you only want to untrack, not delete.

`git reset HEAD-` and `HEAD--` are invalid (“ambiguous argument”). The correct sibling syntax is `HEAD~1` (one commit before HEAD).

5. Branching & checkout

A branch is a movable pointer to a commit. It lets you develop in isolation.

```
git branch          # list branches; * marks current
git branch development # create a branch (does NOT switch to it)
git checkout development # switch to it
# modern equivalent: git switch development
# create + switch in one step: git checkout -b development
```

In your journey you built four branches: `main`, `development`, `staging`. Commits made on one branch are invisible to the others until you merge.

Use `git checkout -b <name>` (or `git switch -c <name>`) to create and switch in a single command.

6. Merging & Resolving Conflicts

Fast-forward vs. real merge

- Fast-forward** When the target branch hasn't diverged, Git just slides the pointer forward — no merge commit. (Your `main` ← `development` merge: `Updating 6d34f7e..16b26e1 Fast-forward`.)
- 3-way merge** When both branches changed, Git creates a **merge commit** using the "ort" strategy. (Your `development` ← `main`: `Merge made by the 'ort' strategy`.)

```
git checkout development
git merge main -m "Merging main into development"
```

Why `-m` was ignored once: on a fast-forward there is no new commit to attach a message to, so Git prints *"Fast-forward (no commit created; -m option ignored)"*.

Handling a merge conflict

When the *same lines* change on both branches, Git stops:

```
git merge staging
# Auto-merging four.txt
# CONFLICT (content): Merge conflict in four.txt
# Automatic merge failed; fix conflicts and then commit the result.
```

Steps to resolve:

1. Open the conflicted file. Git inserts markers:

```
<<<<<< HEAD
your version
=====
their version
>>>>>> staging
```

2. Edit the file to the final desired content and delete the <<< / === / >>> markers.

3. Stage and commit:

```
git add .
git commit -m "Merge conflict solved"
```

After resolving on `development`, merging it back into `staging` and `main` was a clean fast-forward — the hard work only happens once.

7. Inspecting History — `log`, `diff`, detached HEAD

```
git log          # full commits: hash, author, date, message
git log --oneline # compact one-line-per-commit view
```

```
git log --oneline
# d6a5bef (HEAD -> main) update one.txt file
# b76ade1 (staging, development) Merge conflict solved
# 954e59e (origin/main, origin/HEAD) Add new file two.txt ...
```

Labels in parentheses show where each branch/remote pointer sits. `origin/main` is GitHub's copy; when your local is "ahead by N commits" it means you have commits not yet pushed.

Comparing two commits

```
git diff d6a5bef b76ade1
# --- a/one.txt    +++ b/one.txt
# -hello +1      (shows the exact line change)
```

Detached HEAD

Checking out a commit hash directly detaches HEAD:

```
git checkout b76ade1
# You are in 'detached HEAD' state...
```

In detached HEAD, new commits belong to **no branch** and can be lost. To keep experiments: `git switch -c new-branch`. To get back safely: `git checkout main` (or `git switch -`).

8. Working with Remotes — `push`, `fetch`, `pull`

Command	What it does
<code>git push origin main</code>	Upload your local <code>main</code> commits to GitHub.
<code>git fetch</code>	Download remote changes <i>without</i> merging — updates <code>origin/*</code> .
<code>git merge</code>	Merge the already-fetched remote branch into your current one.
<code>git pull</code>	<code>fetch</code> + <code>merge</code> in one step.

```
git push origin main
# 954e59e..d6a5bef  main -> main

git push origin staging
# * [new branch]  staging -> staging
# remote: Create a pull request for 'staging' on GitHub by visiting: ...
```

```
git fetch          # d6a5bef..bac39c2  main -> origin/main
git merge          # Fast-forward into local main
git pull           # fetch + merge together (bac39c2..af51c57)
```

fetch then merge = pull. Use `fetch` + look around when you want to review remote changes before integrating; use `pull` when you trust them and just want to sync.

9. Pull Requests (PRs)

A **Pull Request** is a GitHub feature (not a raw Git command) that proposes merging one branch into another, with review and discussion.

When you pushed a new branch, GitHub printed the exact link to open one:

```
remote: Create a pull request for 'staging' on GitHub by visiting:
remote:   https://github.com/GorkemUtku02/git-journey/pull/new/staging
```

Typical PR flow:

1. Create a feature branch, commit, and `git push origin <branch>`.
2. Open the PR link → choose *base* (e.g. `main`) and *compare* (e.g. `staging`).
3. Reviewers comment; you push more commits to the same branch to update the PR.
4. Click **Merge** (merge commit / squash / rebase) → branch is integrated into `main`.
5. Delete the merged branch; locally run `git pull` to get the merged result.

PRs are the team equivalent of the local merges you practised — same idea, but with review, CI checks, and a record of the discussion.

10. `revert` — Undo a Commit Safely

`git revert` creates a **new commit** that undoes the changes of a previous commit. Unlike `reset`, it does *not* rewrite history, so it is safe on shared/pushed branches.

```
git revert <commit-hash>      # undo one commit, makes a new "Revert ..." commit
git revert HEAD               # undo the most recent commit
git revert --no-commit <hash> # stage the undo but let you commit manually
```

	<code>git reset</code>	<code>git revert</code>
Rewrites history?	Yes	No — adds a new commit
Safe after pushing?	No (needs force-push)	Yes
Use when...	Cleaning up <i>local, unpushed</i> work	Undoing a commit others already have

Rule of thumb: if a commit is already on GitHub and others may have pulled it, use `revert`, not `reset --hard`.

11. `rebase` — Rewrite History Cleanly

`git rebase` moves your branch's commits so they replay *on top of* another branch, producing a straight, linear history (no merge commit).

```
git checkout development
git rebase main          # replay development's commits on top of latest main
```

Merge result

```
A---B---C  main
   \      \
   D---E--M development
(M = extra merge commit)
```

Rebase result

```
A---B---C  main
   \      \
   D'--E'  development
(linear, no merge commit)
```

Interactive rebase — clean up commits

```
git rebase -i HEAD~3      # edit/squash/reorder the last 3 commits
# commands: pick / squash / reword / drop / edit
```

Golden rule of rebase: never rebase commits you have already pushed and shared — it rewrites hashes and breaks everyone else's history. Rebase only your *local, private* commits. (This is why your conflict above was solved with a merge, which is always safe.)

Merge vs Rebase: merge preserves the true history (good for shared branches); rebase makes a cleaner linear history (good for tidying your own feature branch before opening a PR).

12. Quick Command Cheat-Sheet

Goal	Command
Start new repo	<code>git init</code>
Copy remote repo	<code>git clone <url></code>
Check state	<code>git status</code>
Stage all changes	<code>git add -A</code>
Commit	<code>git commit -m "msg"</code>
New branch + switch	<code>git checkout -b <name></code>
Switch branch	<code>git switch <name></code>
Merge a branch in	<code>git merge <name></code>
View history	<code>git log --oneline</code>
Compare commits	<code>git diff <a> </code>
Unstage	<code>git reset</code>
Discard all local changes	<code>git reset --hard</code>
Undo a pushed commit	<code>git revert <hash></code>
Linearize history	<code>git rebase <branch></code>
Upload commits	<code>git push origin <branch></code>
Download (no merge)	<code>git fetch</code>
Download + merge	<code>git pull</code>

13. Mistakes Logged & Lessons Learned

What happened	Why	Fix / lesson
<code>'touch'</code> is not recognized	CMD has no <code>touch</code>	Use <code>echo. > file.txt</code> or <code>type nul > file.txt</code>
<code>'ls'</code> / <code>'dr'</code> not recognized	CMD uses <code>dir</code>	Use <code>dir</code> (or run Git Bash for Unix commands)
<code>gti clone</code> , <code>git chechkout</code> , <code>add.</code>	Typos	Read Git's "most similar command" hint
<code>git reset HEAD-</code> failed	Wrong syntax	Use <code>HEAD~1</code> for the previous commit
<code>git rm four.txt</code> blocked	File had local edits	Use <code>--cached</code> (untrack) or <code>-f</code> (force delete)
<code>git add .</code> missed <code>../</code> files	It's relative to current folder	Use <code>git add -A</code> for the whole repo
Commit typo "smoe changes"	Message typo, already committed	Fix latest message with <code>git commit --amend</code> (before pushing)

